Tolerant frames II

What the bound proofs of a text-layout library need

TScert working note

13 November 2026

Abstract

The text-layout library pretext verifies under TScert only in a rewritten form. The rewritten form has 422 sites where an index read carries a check that the original source does not have. This note asks, for each site, what the proof of its bound needs in the logic of the first paper, *Tolerant frames*. The 215 sites that need a capability fall under six capabilities. Two are derivable in the first paper: postconditions that say by how much a length changed, and a group of rules whose instances TScert does not derive. A third, finer regions for what a call may touch, is derivable for two of its three patterns and needs regions by kind of location for the third. Two need small extensions: bounded effect variables for a callback passed in, and contracts that hold of a location or of a type for its whole life. The last needs five richer term forms. Each extension is a definition with its meaning, one example, and the statement that makes it sound. One worked program shows four of the capabilities at marked lines. The analysis's classification changes for two patterns. Every statement carries one of four marks. [paper] names the definition, axiom, rule or theorem of the first paper that derives it. [observed] names the test program on which TScert's verifier gave the stated verdict. [proved on paper] marks a statement proved in this note. [conj.] marks a conjecture.

1 The question

The library pretext lays out text: it measures segments of a string into parallel arrays, and walks those arrays to break lines. Its source verifies under TScert only after rewrites. A *bound rewrite site* is an index read `x[k]` that the rewritten source writes as `x[k] ?? missing()` or guards by a `RangeError`, because no proof of $0 \le k < |x|$ is found there. This note asks which capability of a logic each such proof lacks. The logic is the one of the first paper, *Tolerant frames: a modal account of fact preservation without ownership* (11 November 2026), written TF below; "TF §n" names its section n. The sites and their minimal programs are those of an analysis of pretext's rewritten source at commit `9268edb1`; every site count below comes from that analysis.

The count. Of the 422 bound rewrite sites, 215 are *blocked*: the proof at the site itself needs a capability. 204 are *locally supported*: the proof at the site needs nothing new, given facts established at a builder or a caller. Of these, 39 have the shape of a verified program [observed] and 165 are [conj.]. 3 sites are not placed (Section 12). The 215 blocked sites fall under six capabilities:

	capability	patterns	sites	in TF
1	the effect of a callback passed in	P1	50	no: bounded effect variables
2	posts that say by how much a length changed	P2, P8, P14	45	yes: CALL, two-state posts
3	finer regions for what a call may touch	P3, P4, P6	49	P3, P4 yes; P6 no: kind regions
4	facts of a location or a type for good	P5, P15, P16, P20	25	no: contracts
5	richer terms	P9, P10, P12, P17, P19	24	no: five term forms
6	rules TF has and TScert lacks	P7, P11, P13, P18, P21	22	yes
	total		215	

A *pattern* Pn is the smallest program shape whose bound proof lacks one capability. The labels P1 to P21 are the analysis's. Each section below takes one capability. For each pattern it gives the minimal program, the fact the proof needs in TF's assertion language, where TF stops, and the smallest extension. Section 10 gives the class of each pattern as this note finds it. Section 11 states what makes each extension sound in TF's model.

2 The notions used

This section recalls the notions of TF that later sections use, each with its definition, one example, and its place in TF.

Heaps and kinds (TF §2) A heap is a finite map $h : Loc \rightharpoonup Val \uplus Store$ with $Loc = \mathbb{N}$ and $\mathrm{dom}\, h = \{0, \ldots, |h|-1\}$. Its domain is partitioned by kind: $\mathrm{dom}\, h = F \uplus C \uplus E \uplus S$, the field locations, the cells, the element cells and the store locations. The first three hold values and form the *value* class; S holds stores and forms the *store* class. A cell is the location of a module binding, or of a local binding that a closure writes or sees change; every other local binding is a variable of the environment $\rho : Var \rightharpoonup Val$. Example: after `const o = { n: 1 };` the object `o` is the map $\{\mathtt{n} \mapsto l\}$ for one $l \in F$, and `o` is a variable.

Stores and lengths (TF §2) $Store = (Loc \uplus \{\circ\})^*$. For $\sigma \in Store$, $|\sigma|$ is its length and $cells(\sigma) = \{c_i \mid c_i \neq \circ\}$. An array is an object with one distinguished field whose location $s \in S$ holds its store. Example: `const a = [10, , 30];` gives $h(s) = \langle l_0, \circ, l_2 \rangle$, length 3, $l_0, l_2 \in E$.

Contracts and immutable locations (TF §2, §3) A world W maps each location to its *contract*, the set of contents it may hold. A location is immutable when its contract admits one value. TF has two kinds: a field declared `@stable`, and a typed array's length, written view(w). Example: in `type Runs = { /** @stable */ readonly openBrackets: number[] }` the field `openBrackets` of each `Runs` object always holds the same array.

Admissible transitions (TF §3) $h \rightarrow_\rho h'$ when h' is the heap of a state whose world extends that of h: every location keeps its contract, and h' meets it. Every run of typed code is admissible [paper: TF §3, *Admissible transitions*]. Its properties are (T1) preorder, (T2) growth $|h| \leq |h'|$, (T3) fixed objects: an object's keys and field locations do not depend on the heap, and (T4) immutable locations: a location whose contract admits the single value u holds u in h and h'. Example: `o.n = 2` on the object above is admissible when the contract of l is "a number".

Changes (TF §3) A change δ is a family of preorders δ_b on contents; keep $\sqsubseteq$ init $\sqsubseteq$ scalar $\sqsubseteq$ grow $\sqsubseteq$ any. grow$_b$ relates two stores when the second has a length at least the first's, keeps every dense prefix, and lists no old location the first does not list. Example: grow$_6$ relates $\langle 4, \circ, 5 \rangle$ to $\langle 4, \circ, 5, 6 \rangle$ and not to $\langle 4 \rangle$.

Regions (TF §3) A path $\pi = x.f_1 \ldots f_k$ is a variable followed by field names. A region r denotes a set $r(\rho, h)$ of locations: all-stores denotes S; all-values denotes $F \cup C \cup E$; π.store denotes the store location of the array π reads; $\pi[*]$ denotes it with the cells its store lists; $\pi.f$ denotes one field location. Example: for `a` above, $a[*]$ denotes $\{s, l_0, l_2\}$.

Effects and their action (TF §3) An effect Φ is a finite set of entries $r : \delta$. $h \xrightarrow{\Phi}_\rho h'$ when $h \rightarrow_\rho h'$ and every location $l < |h|$ holds what it held or lies in a region $r(\rho, h)$ of an entry $r : \delta$ and changed within $\delta_{|h|}$. $\top$ = {all-values : any, all-stores : any}. Φ is *rigid* when its action is transitive; effects over all-values and all-stores are rigid. Example: the effect of `out[j] = v` is {all-values : any, all-stores : grow}.

Assertions (TF §4) Atoms are linear inequalities over variables, lengths $|\pi|$ and view(π); equalities $\pi \doteq v$; dense(x, n); kinds of contents; and every(z, t, φ), each element of z below index t satisfies φ, a formula of that one element. Assertions add $\wedge$, $g \vee F$ with g invariant, $\exists$, $\pi \,\#\, \pi'$ (different store locations), apart(x, y) (no common element cell), fresh(x) and the modality. Example: every$(a, 1, v > 0)$ holds of `[10, , 30]`.

The modality (TF §4) $\rho, m, h \models \Box_\Phi F$ when F holds in every h' with $h \xrightarrow{\Phi}_\rho h'$. F is *invariant* when $F \vdash \Box_\top F$; rigid$(\pi) \triangleq \exists v.\, \Box_\top(\pi \doteq v)$. Example: $3 \leq |a|$ entails $\Box_{\text{all-values:any, all-stores:grow}}(3 \leq |a|)$, and does not entail $\Box_{a.\text{store:any}}(3 \leq |a|)$.

Support, respect, tolerant frame (TF §5) A support S of F is a set of pairs $r : \delta$ such that F survives every admissible transition that changes each location of each r within its δ. $\Phi \triangleleft_K S$ when each entry $r' : \gamma$ of Φ and each pair $r : \delta$ of S have $\gamma \sqsubseteq \delta$ or regions disjoint under K. Then $K \wedge F \vdash \Box_\Phi F$ [paper: Theorem *Tolerant frame*]. Example: $\{x.\text{store} : \text{grow}\}$ supports $n \leq |x|$.

Axioms (TF §5) (Grow) keeps a lower bound of a length under {all-values : any, all-stores : grow}; (Dense) a dense prefix under the same effect; (Sep) a fact of $|x|$ under {all-values : any, y.store : any} given $x \,\#\, y$; (Every) an every over z under $\{x[*] : \text{any}\}$ given apart(x, z); (Init) a length under {all-values : init}; (Rigid) a fact of a variable's array for the array at a rigid path; (View) a fact of view(w) under $\top$. Example: (Grow) gives $n \leq |x| \vdash \Box_{\text{all-values:any, all-stores:grow}}(n \leq |x|)$.

Triples and rules (TF §6) $\{P\}\ c\ \{Q \mid Q^e\}$ is partial correctness; Q is two-state, with old(e) the value of e where c started. The rules are Frame, Frame$^+$, Loop, Call, Alloc, Seq-Eff, Fresh-Eff. Example: `x.push(v)` has the post $|x| = \mathsf{old}(|x|) + 1$.

Checked transfer (TF §7) An edge e of a control-flow graph carries an effect Φ_e, a sound two-state post Q_e, a condition and a renaming. A fact crosses e when it is derivable under $\Box_{\Phi_e}$; the rest is derived from Q_e. Example: the node $j' = j + 1$ has the post $j' = j + 1$.

The sets and maps of the account are these.

> **Definition 1** (Sites)**.** R is the set of bound rewrite sites, $|R| = 422$. It is partitioned as $R = B \uplus L \uplus U$: the blocked sites B, $|B| = 215$; the locally supported sites L, $|L| = 204$; the sites not placed U, $|U| = 3$. $\mathcal{P} = \{\mathrm{P1}, \ldots, \mathrm{P21}\}$ is the set of patterns. The map $p : B \to \mathcal{P}$ gives a blocked site its first blocker, the first capability its own read lacks. The map $\kappa : \mathcal{P} \to \{1, \ldots, 6\}$ gives a pattern its capability.

Example: the read `starts[i + 1]` of `measureAnalysis` at line 378 of `prepare.ts` is a site $r \in B$ with $p(r) = \mathrm{P4}$ and $\kappa(\mathrm{P4}) = 3$.

> **Definition 2** (Classes)**.** Let F be the fact a pattern's bound proof needs, and c the step it must cross. The pattern is in one class:
>
> **(a) not stated** No assertion of TF states F. Example: $|\mathit{rows}| = \mathit{states} \cdot \mathit{width}$, a product of two variables.
>
> **(b) not kept** F is an assertion, and for no effect Φ that TF can state for c is $\Box_\Phi F$ derivable. Example: an every over a `number[]` across a call that writes a module cell, whose only TF effect is {all-values : any}.
>
> **(c) kept in TF** F is an assertion and its proof across c is derivable in TF, from posts sound for the steps involved. Example: a length across a call whose two-state post states the length.
>
> **(d) a contract** F holds because of the contract of a location or an invariant of a type, beyond the single-value contracts of TF. Example: the length of a module constant array literal.

Two notions beyond TF are used throughout.

> **Definition 3** (String length)**.** For a variable x that holds a string, strLength$(x) \in \mathbb{N}$ is the number of UTF-16 code units of $\rho(x)$. Linear terms may contain strLength(x).

A string is a scalar, so strLength(x) reads no location: its support is $\emptyset$, and every atom over variables and string lengths is invariant. Example: after `const s = "ab"`, strLength$(s) = 2$ holds after every step. TF's atoms have array lengths and view only. TScert has this term [observed: `t08_strings.ts`, `goodCode` verified].

> **Definition 4** (Paths from cells)**.** A path may start at a binding b modelled as a cell $l_b \in C$. Its read starts from $h(l_b)$.

Example: for the module binding `let bmp = null`, the path *bmp* reads $h(l_{bmp})$. TF uses this reading for the cell `c` of its write-once cache pattern (TF §8) and does not define it. A fact through such a path reads l_b, so it survives an effect disjoint from l_b, or any effect when a contract fixes l_b (Definitions 15 and 17).

3 A worked program

The program below builds two parallel arrays and walks them. It is this note's own, in the shape of pretext's builder and walker. A mark (n) names the capability whose proof the line needs.

```
1  type Lists = { /** @stable */ readonly widths: number[];
2                 /** @stable */ readonly kinds: number[] };
3  const KINDS: readonly string[] = ["text", "space", "break"];
4  let calls: number = 0;
5  function measure(text: string): number { calls++; return text.length; }
6  function append(lists: Lists, texts: string[]): void {
7    for (let i = 0; i < texts.length; i++) {
8      const w = measure(texts[i]!);                                    (3)
9      lists.widths.push(w);
10     lists.kinds.push(texts[i] === " " ? 1 : w === 0 ? 2 : 0);        (6)
11   }
12 }
13 function build(texts: string[]): Lists {
14   const lists: Lists = { widths: [], kinds: [] };
15   append(lists, texts);                                              (2)
16   return lists;
17 }
18 function walk(lists: Lists, visit?: (i: number) => void): number {
19   const n = lists.widths.length; let x = 0;
20   for (let i = 0; i < n; i++) {
21     x = lists.kinds[i]! === 2 ? 0 : x + lists.widths[i]!;
22     visit?.(i);                                                      (1)
23   }
24   return x;
25 }
26 export function lastKind(texts: string[]): string {
27   const lists = build(texts);
28   const x = walk(lists, undefined);                                  (1)
29   const n = lists.kinds.length;
30   return n === 0 ? "" : KINDS[lists.kinds[n - 1]!]! + x;             (4)
```

The heap. Number the locations in allocation order, the module first. At line 28 of `lastKind(["ab", " "])`, before the call of `walk`, one heap h_{28} the program can have is:

location	kind	holds	
0	C	k	the cell of `KINDS`
1	S	$\langle 2,3,4\rangle$	the store of `KINDS`
2, 3, 4	E	`"text"`, `"space"`, `"break"`	its elements
5	C	2	the cell of `calls`
6	S	$\langle 7,8\rangle$	the store of `texts`
7, 8	E	`"ab"`, `" "`	its elements
9	S	$\langle 13,15\rangle$	the store of `lists.widths`
10	S	$\langle 14,16\rangle$	the store of `lists.kinds`
11	F	W	the field `lists.widths`
12	F	K	the field `lists.kinds`
13, 15	E	2, 1	`lists.widths[0]`, `lists.widths[1]`
14, 16	E	0, 1	`lists.kinds[0]`, `lists.kinds[1]`

Here k, W and K are the arrays whose store locations are 1, 9 and 10. $C = \{0,5\}$, $S = \{1,6,9,10\}$, $F = \{11,12\}$, and E is the rest. The environment holds `texts` and *lists* $= \{$`widths` $\mapsto 11,$ `kinds` $\mapsto 12\}$. The fields 11 and 12 are `@stable`, so rigid(*lists.widths*) and rigid(*lists.kinds*) hold [paper: TF §4, *Invariant assertions and rigid paths*].

The facts. The read at line 30 needs, at line 28, the three facts

$F_1 = \mathsf{every}(\mathit{lists.kinds},\ \lvert\mathit{lists.kinds}\rvert,\ 0 \le v < 3)$	the kinds index `KINDS`,
$F_2 = (\lvert\mathit{KINDS}\rvert = 3)$	the length of the module table,
$F_3 = (\lvert\mathit{lists.widths}\rvert = \lvert\mathit{lists.kinds}\rvert)$	the arrays are parallel.

F_3 is `walk`'s precondition, and F_1 and F_2 must survive the call at line 28. Each later section returns to its marked line: line 22 and line 28 in Section 4, line 15 in Section 5, line 8 in Section 6, line 30 in Section 7, and line 10 in Section 9.

The verdicts. As printed, with no annotation beyond the types, TScert verifies `measure`, `append` and `build`: they push and read `texts[i]` below its length, and no read of theirs needs a fact from elsewhere. It refuses `walk` at the read `lists.kinds[i]` of line 21, which needs F_3, a precondition the listing does not write. It refuses `lastKind` at the read `KINDS[lists.kinds[n - 1]!]` of line 30, which needs F_1 and F_2 [observed: branch `single-pass` at `b7c8a559`, 13 November 2026]. Line 4 writes the type of `calls` because TScert requires a module `let` to state its type.

4 The effect of a callback passed in

4.1 P1: a visitor of unknown effect between reads (50 sites)

Program. `function walk(xs: number[], visit: Visitor | undefined): number` reads `xs[i]` in a loop and calls `visit(s)` when `visit !== undefined`. The function `bad` calls `walk(xs, undefined)` and then reads `xs[i]` under the precondition $0 \le i < |xs|$ [observed: `t02_optional_callback.ts`].

Fact. After the call, $i < |xs|$. In pretext's walker `walkPreparedComplexLines`, with c the segment count read at entry, each loop head needs $c \le |widths| \wedge 0 \le cursor.segmentIndex < c$ across the call `onLine?.(...)`.

Where TF stops. The parameter's type states no effect, so the effect of a call of `visit` is $\top$, and so is the effect of `walk`. The transition that sets `xs.length = 0` is bounded by $\top$ and breaks the fact. TF states one closed bound per callback parameter: an effect has no variables (TF §10, *Effects are closed*). A closed bound keeps lower bounds of lengths: with the bound $\Psi = \{\text{all-values} : \text{any},\ \text{all-stores} : \text{grow}\}$, $i < |xs|$ survives [paper: (Grow)] [observed: `t02_optional_callback.ts`, `goodBounded` verified]. Two things remain out of reach. Inside the walker, the fact $0 \le cursor.segmentIndex$ reads a field location; every closed bound that admits the application's visitors contains all-values : any, which covers the field, and no TF region excludes one object's fields from all-values. At a caller passing no visitor, the walker calls nothing, yet under a closed bound its effect is still Ψ, so every fact over value locations and every upper bound of a length is lost at the call. The first needs a region; the second needs an effect variable.

> **Definition 5** (Complement and object regions)**.** For a path π that reads an object v, the region $\pi.*$ denotes $\{v(f) \mid f \in \operatorname{dom} v\}$, the field locations of v. For regions r_1, r_2, the region $r_1 \setminus r_2$ denotes $r_1(\rho, h) \setminus r_2(\rho, h)$.

Example: if $cursor = \{\texttt{segmentIndex} \mapsto 20,\ \texttt{graphemeIndex} \mapsto 21\}$, then $cursor.*$ denotes $\{20, 21\}$, and all-values $\setminus cursor.*$ denotes every value location but 20 and 21. Under the bound $\Psi_{onLine} = \{\text{all-values} \setminus cursor.* : \text{any},\ \text{all-stores} : \text{grow}\}$ the fact $0 \le cursor.segmentIndex < c$, of support $\{cursor.segmentIndex : \text{keep}\}$, survives by the Tolerant frame theorem: one entry is disjoint from the field, and the other has the store class.

> **Definition 6** (Bounded effect variable)**.** Let p be a parameter of function type of a function f. An *effect variable* ε_p stands for the effect of the value p holds. A signature of f has the form
>
> $$\forall \varepsilon_p \sqsubseteq \Psi.\quad \{P\}\ f(\vec{p})\ \{Q\} : \Phi \cup \varepsilon_p,$$
>
> where the *bound* Ψ is a closed rigid effect over class regions and regions at rigid paths of f's parameters. Inside f a call of p has the effect ε_p. At a call of f the variable is instantiated: to $\emptyset$ when the argument is `undefined`, and to Ψ' when the argument is a function of effect Ψ', with the obligation $\Psi' \sqsubseteq \Psi$. The effect of the call is $\Phi \cup \Psi'[\vec{x}/\vec{p}]$.

The rule it gives at a call, Call-Var, is Call at the instance:

$$\textsc{Call-Var}\ \frac{\begin{array}{c} f(\vec{p}) : \forall \varepsilon_p \sqsubseteq \Psi.\ \{P\}\ f\ \{Q\} : \Phi \cup \varepsilon_p \\ x_p \text{ holds a function of effect } \Psi', \text{ or } \texttt{undefined} \text{ and } \Psi' = \emptyset \qquad \Psi' \sqsubseteq \Psi \end{array}}{\{P[\vec{x}/\vec{p}] \wedge \square_{(\Phi \cup \Psi')[\vec{x}/\vec{p}]} R\}\ y = f(\vec{x})\ \{Q[\vec{x}/\vec{p}] \wedge R\}}$$

Inside f, a fact survives a call of p for every admitted ε_p exactly when it survives Ψ, by law (c) of TF's Proposition *Laws* and because Ψ itself is admitted. So the variable adds nothing to the walker's own proof; it makes the caller's effect exact.

The worked program. At line 22, `walk` : $\forall \varepsilon_{visit} \sqsubseteq \{\text{all-values} : \text{any},\ \text{all-stores} : \text{grow}\}.\ \emptyset \cup \varepsilon_{visit}$. The loop facts $n \le |lists.widths|$ and $n \le |lists.kinds|$ are lower bounds at rigid paths and survive Ψ [paper: (Grow), (Rigid), Loop]. At line 28 the argument is `undefined`, so the call has the effect $\emptyset$, and F_1 and F_2 hold after it by Frame [proved on paper: Section 11]. Without the variable, the effect at line 28 is Ψ. The visitor `i => { lists.kinds[0] = 7; }` is bounded by Ψ, it changes location 14 of h_{28} to 7, and line 30 then reads `KINDS[7]`, which is `undefined`.

Sites. 47 sites lie inside the two walkers that call `onLine`: `walkPreparedLinesRaw` (310) and `walkPreparedComplexLines` (46 sites, 584 to 1085). They need a closed bound with Definition 5. 3 sites lie at callers that pass no visitor: `countSteppedLines` (407) and `prepareRichInline` (368, 496). They need Definition 6.

Evidence. `bad` is refused at `xs[i]`, and `goodBounded` and `goodPlain` are verified [observed: `t02_optional_callback.ts`]. `flagsWalk`, which reads a `Uint8Array` below a count taken at entry across a visitor of unknown effect, is verified by (View); `widthsWalk`, the same loop over a `number[]`, is refused [observed: `t13_view_callback.ts`].

> *Remark* 7 (Classification of P1). The analysis gives P1 one definition. This note finds two needs. The 47 walker sites are class (b) for want of Definition 5: the fact is stated, and TF can state no bound that admits the application's visitors and keeps the cursor's fields. The 3 caller sites are class (b) for want of Definition 6. The bound on a visitor that the application passes is a trust decision in both (Section 12).

5 Posts that say by how much a length changed

These patterns are in TF. A step's post may be two-state (TF §6, *Triples*), and Call carries the callee's post $Q[\vec{x}/\vec{p}]$ to the caller [paper: Call]. An exact post constrains the written location only; every other fact crosses the step by Frame (TF §6, *Exact posts*).

5.1 P2: growth by a known count across a callee (31 sites)

Program. `measure(lists, n)` pushes n values onto `lists.widths`, a `@stable` field, with the effect {all-values : any, *lists.widths*[∗] : any}. The caller calls `measure(lists, 10)` in a loop on `j += 10` with `j + 10 <= cap`, and reads `flags` at the last index of `lists.widths`, where `flags = new Uint8Array(cap)`. The program as TScert verifies it [observed: the case `bounds-pattern-known-growth-refused`, verified on the branch `single-pass`, 13 November 2026]:

```
1  type Lists = { /** @stable */ readonly widths: number[] };
2  /** @requires safeInteger(n) && 0 <= n && n <= 1000 && 0 <= lists.widths.length
3   *  @ensures lists.widths.length === old(lists.widths.length) + n */
4  function measure(lists: Lists, n: number):
5      Modifies<{ values: "any"; stores: "keep"; "lists.widths[*]": "any" }, void> {
6    /** @invariant safeInteger(k) && safeInteger(n) && 0 <= k && k <= n
7     *            && lists.widths.length === old(lists.widths.length) + k */
8    for (let k = 0; k < n; k++) lists.widths.push(k);
9  }
10 /** @requires safeInteger(cap) && 0 <= cap && cap <= 100000 */
11 export function grown(cap: number): number {
12   const widths: number[] = [];
13   const lists: Lists = { widths };
14   const flags = new Uint8Array(cap);
15   let s = 0;
16   /** @invariant safeInteger(j) && 0 <= j && safeInteger(cap)
17    *            && 0 <= lists.widths.length && lists.widths.length <= j && j <= cap */
18   for (let j = 0; j + 10 <= cap; j += 10) {
19     measure(lists, 10);
20     s += flags[lists.widths.length - 1]!;
21   }
22   return s;
23 }
```

Line 3 is the two-state postcondition: `old(e)` is the value of `e` where `measure` was entered. The read under proof is at line 20. Each other annotation is needed, for these reasons.

Line 2, `n` a nonnegative safe integer Without it the postcondition is false: `measure(lists, 1.5)` pushes two values, and a negative `n` pushes none.

Line 2, `0 <= lists.widths.length` It states that the length is defined at entry. In TScert's model this does not follow from the type `Lists`: a value meets its type only to the depth of the step budget [observed: `measure` is refused at its entry without it].

Line 5, the effect The caller keeps its facts about `flags` across the call because the call changes no store but `lists.widths`.

Lines 6–7 and 16–17, the loop invariants Rule Loop needs an invariant at each loop head.

Line 20, the read through `lists.widths` With the read written `flags[widths.length - 1]`, through the local name, `grown` is refused at the read [observed]. TF derives that form too, from $lists.widths \doteq widths$ and $\mathsf{rigid}(lists.widths)$; why TScert does not is not established.

Fact. The loop invariant $|widths| = j$, with `lists.widths` the array `widths`. It needs the callee's post

$$\{\mathsf{true}\}\ \texttt{measure}(lists, n)\ \{|lists.widths| = \mathsf{old}(|lists.widths|) + n\}.$$

In TF. Call gives $|lists.widths| = j + 10$ after the call; $\mathsf{rigid}(lists.widths)$ and $lists.widths \doteq widths$ give $|widths| = j + 10 \le cap = \mathsf{view}(flags)$, and (View) keeps $\mathsf{view}(flags) = cap$. The read at $j + 10 - 1$ is in bounds [paper: Call, (Rigid), (View), Loop]. Class (c).

The worked program. At line 15, `append` has the post

$$|lists.widths| = \mathsf{old}(|lists.widths|) + |texts| \ \wedge\ |lists.kinds| = \mathsf{old}(|lists.kinds|) + |texts|.$$

Both arrays are empty at line 14 [paper: Alloc], so F_3 holds after line 15. A one-state post of `append` states lower bounds only, and F_3 does not follow from them.

Sites. 30 in `prepareRichInline`, where each read or guarded write at `at = widths.length` needs $|widths| \le \mathsf{view}(flags)$ after `measureAnalysis` appended `to - from` segments; 1 in `measureAnalysis` (631).

Evidence. The program above is verified. With the callee growing the list by more than the loop's step, the read is refused, and the program is unsafe under Node [observed: the case `invalid-bounds-pattern-known-growth`]. With the pushes written in the caller's own body, the program is verified, each `push` having the post $|widths| = \mathsf{old}(|widths|) + 1$ [observed: the case `bounds-pattern-known-growth`].

5.2 P8: the length after a write at the length (10 sites)

Program. `xs[k] = 1; return xs[k]!;` under $0 \le k \le |xs|$ [observed: `t17_write_len.ts`].

Fact. After the write, $k < |xs|$ and $\mathsf{dense}(xs, k+1)$ when $\mathsf{dense}(xs, k)$ held before.

> **Proposition 8** (Write at the length)**.** *[proved on paper] For an array x and $0 \le k \le |x| < N$,*
>
> $$\{\mathsf{true}\}\ x[k] = v\ \{|x| = \max(\mathsf{old}(|x|), k+1)\ \wedge\ (\mathsf{old}(\mathsf{dense}(x,k)) \Rightarrow \mathsf{dense}(x, k+1))\}.$$

Proof. By TF's table of writes (TF §2). If index k is present, the write changes its element cell and leaves the store. If k is a hole below the length, the store is filled at k and keeps its length. If $k = |x|$, the store gains one new cell at index k. In each case index k is present after the write, every other index keeps its presence, and the length is $\max(\mathsf{old}(|x|), k+1)$. □

In TF this post is a sound two-state post of the step, which checked transfer admits (TF §7). TF does not state it. Class (c). A builder that appends by index, `widths[at] = w` with `at` the current length, keeps $at \le |widths|$ at its loop head with it.

Sites. 9 in `measureAnalysis` (`prepare.ts`) and 1 in `prepareRichInline` (646).

Evidence. `readWritten`, `readBefore` and `readBelow` are verified [observed: `t17_write_len.ts`, branch `single-pass` at `b7c8a559`]. `append` and `appendNoCall`, builders that append by `widths[at] = mi`, are verified [observed: `t16_supported.ts`, same branch]. TScert derives this post.

> *Remark* 9 (The post of P8). The analysis writes this post with the conjunct $x[k] \doteq v$. That conjunct reads an element at an index, which TF's paths do not express (Definition 22 adds it). The bound proofs of the 10 sites need only the length and the density, which TF states.

5.3 P14: length contracts of string and array natives (4 sites)

Program. `const normalized = text.replace(/ +/g, " ");` followed by reads of `breaks = new Uint8Array(text.length + 1)` at indices below `normalized.length` [observed: `t08_strings.ts`].

Fact. strLength(*normalized*) ≤ strLength(*text*), the post of the native step. With Definition 3 it is invariant once stated. What is missing is the post itself. The posts the sites need are:

`r = s.replace(e, t)`, each match no shorter than t	strLength(r) ≤ strLength(s)
`r = s.split(d)`	$1 \le \lvert r \rvert$
`r = xs.map(f)`	$\lvert r \rvert$ = old($\lvert xs \rvert$)
`r = w.subarray(b, e)`, $0 \le b \le e \le$ view(w)	view(r) = $e - b$

The post of `map` holds whatever `f` does: `map` reads the length once at its start (TF §6, Proposition *Reliance*). Each post is an assumption about a native, a trust decision (Section 12). Class (c).

Sites. `getGeckoLineBreaks` (684), `getWebKitLineRules` (784, a `split`), and `getFontData` (145, 146, a `map` of a ten-element literal). 33 locally supported sites depend on the `replace` and `slice` posts of `analyzeText`.

Evidence. `bad` is refused at its loop's entry edge; `goodSlice`, a `slice` with an exact length contract, is verified [observed: `t08_strings.ts`].

> *Remark* 10 (String lengths in P11 and P14). The facts of P14's minimal program, of site 684 and of P11 are about string lengths, which TF's atoms do not express. They are class (c) with Definition 3, a term of empty support. The other three sites of P14 are about array lengths and are class (c) in TF.

6 Finer regions for what a call may touch

A fact survives a step when the step's effect respects a support of the fact. Two of the three patterns here are instances of the Tolerant frame theorem; one needs a new region.

6.1 P3: a quantified fact across a write outside its array (19 sites)

Program. A loop over `items: Int32Array` writes `fragment.width = s` of a local object and reads `widths[items[i]!]` [observed: `t03_every_fresh_field.ts`].

Fact. F = every(*items*, view(*items*), $0 \le v < \lvert widths \rvert$) across the write, whose effect is {*fragment.width* : any}.

In TF. S = {*items*[∗] : keep, *widths*.store : grow} supports F: the fact reads the store and the cells of `items`, and a lower bound of the length of `widths`; view(*items*) is immutable. The region *fragment.width* denotes a location of F-kind, and the two regions of S denote locations of $S \cup E$. They are disjoint under K = true, so $F \vdash \Box_{\mathit{fragment.width}:\mathrm{any}} F$ [paper: Theorem *Tolerant frame*]. Class (c).

Sites. 12 in `createLine` (`rich-inline.ts`), between a field write of `fragment` and `fragments.push(fragment)`; 7 in `setEmptyObjectFacts`, across element writes of `spaces`, a `number[]`, and `returns`, a `Uint8Array`. For the 7, the effect is {*spaces*[∗] : any}, and respect needs K = apart(*spaces*, z) for each array z the facts read. TF gives apart by Alloc at the builder, where the arrays are created; the facts must then carry it to these sites [conj.]. Different element types give no apart in TF.

Evidence. `bad` is refused at `widths[items[i]!]`, also with the fact as a loop invariant; `good`, the loop without the write, is verified [observed: `t03_every_fresh_field.ts`, `t03b_every_inv.ts`].

> *Remark* 11 (Two sites of P3). The read at line 1185 of `createLine` (2 sites) also needs every(*fragments*, k, *fragments*[i].*itemIndex* < n), a fact about a field of each element. TF lists that form outside its language (TF §8). So these 2 sites are class (a) in TF, and Definition 21 states the fact. The other 17 sites of P3 are class (c).

6.2 P4: an upper bound of a length across a call growing stores (18 sites)

Program. `note(out, v)` pushes onto `out`. A loop with test `i + 1 < a.flags.length` calls `note(out, i)` and reads `a.starts[i + 1]`, where `a.flags` and `a.starts` are `@stable`. The precondition states $\lvert a.flags \rvert = \lvert a.starts \rvert$, *a.flags* # *out* and *a.starts* # *out*. The two separations are part of the pattern. Without them the program is unsafe: called with `out` the array `a.flags`, each push lengthens `a.flags`, the test stays true, and the read passes the end of `a.starts` [observed: the case `invalid-bounds-pattern-grow-upper`, under Node].

Fact. $\lvert a.flags \rvert \le \lvert a.starts \rvert$ across the call.

In TF. Under the effect {all-values : any, all-stores : grow} the fact is lost, since that effect lets every store grow, `a.flags` included (TF §7, the full version of its Example 1). The callee's writes are to one array its parameter

names, so TF states for it the effect {all-values : any, *out*.store : any}. With $K = a.flags \# out \wedge a.starts \# out$ the fact survives by (Sep) at the rigid paths [paper: (Sep), (Rigid)]. Class (c), given the callee's regional effect.

Sites. 4 in chain 1 (`starts[i + 1]` under `i + 1 < flags.length`), 10 in `measurement.ts` (the advances list's exact length across measuring calls), 4 in `prepareRichInline`. Whether each callee's store writes are at paths of its parameters, so that a regional effect states them, is not tested [conj.].

Evidence. `goodRegion`, the program above with the regional effect and the separations at the paths `a.flags` and `a.starts`, is refused, and `goodLower` (a lower bound) is verified [observed: `t01_grow_equal.ts`]. The variant with no separation stated is refused, and that refusal is correct. `slots`, the regional effect with the arrays as parameters, is verified [observed: `t01b_region.ts`]; `paths` is refused and `slots` verified [observed: `t01c_lists.ts`]. TScert keeps the fact for variables and not for rigid paths.

> *Remark* 12 (Classification of P4). The analysis places P4 in class (b), since the fact is lost under the effect `stores: "grow"`. In TF a regional effect states the callee exactly, and under it the fact is kept. This note places P4 in class (c).

6.3 P6: a fact over value locations across a call changing values (12 sites)

Program. A loop keeps every($starts, |starts|, 0 \le v <$ view($offsets$)) across `touch(c)`, stated with the effect {all-values : any} [observed: `t14_every_call.ts`, `badAny`].

Where TF stops. all-values contains the element cells of `starts`, and no axiom keeps the fact under {all-values : any}. The function `touch` writes `c.value`, so TF states for it the effect {$c.value$: any}, under which the fact is kept by kinds, as for P3. The capability is needed by a callee that writes locations no path from its parameters reaches: module cells, and fields of objects it does not receive. Its only TF effect is {all-values : any} or $\top$.

> **Definition 13** (Kind regions)**.** all-fields$(\rho, h) = F$, all-cells$(\rho, h) = C$ and all-elements$(\rho, h) = E$. So all-values denotes the union of the three. Kind regions may also appear in the bound of an effect variable (Definition 6).

Example: `let hits = 0; function touch(): void { hits++; }` has the effect {all-cells : any}. That effect respects the support {$starts[*]$: keep} under true, since C is disjoint from $S \cup E$, so the every survives the call [conj.: this program is not run].

The worked program. At line 8, `measure` writes the cell of `calls`. Its effect in TF is {all-values : any}, which loses the loop invariant every($lists.kinds, |lists.kinds|, 0 \le v < 3$). With Definition 13 its effect is {all-cells : any}. The invariant's support is {$lists.kinds$: keep, $lists.kinds[*]$: keep}, of kinds F, S and E, so it survives. The lengths in the invariant survive too, by the same argument.

Sites. 5 in `prepareRichInline` (`offsets[analysis.starts[...]]` across `measureAnalysis`), 3 in `materializeRichInlineLineRange`, 1 in `getFollowingSpaceTail`, 3 in `getHanKerningTrims`. The facts read `analysis.starts` through a field; under an entry all-fields : any they also need rigid($analysis.starts$), which `@stable` gives. The 3 sites of `getHanKerningTrims` read a field of a fresh object `out` that the callees do not receive; kind regions keep that fact when the callees write no field location [conj.]. Which locations pretext's measuring callees write is not checked [conj.].

Evidence. `badAny` is refused; `badInit`, whose callee has the effect {all-values : init}, is verified, as (Init) predicts [observed: `t14_every_call.ts`].

> *Remark* 14 (Classification of P6). The minimal program of the analysis is class (c) in TF, by the effect {$c.value$: any}. The pattern is class (b) for callees whose written locations no TF region names, as in the module-cell program above. This note keeps P6 in class (b) on the analysis's reading that pretext's measuring callees are of that kind [conj.].

7 Facts of a location or of a type for good

By (T4) a fact that reads only immutable locations is invariant. TF's immutable locations are `@stable` fields and typed-array lengths; the patterns here need more.

7.1 P15: a module constant array (4 sites)

Program. `const KINDS: readonly string[] = ["a", "b", "c", "d"];` read at `KINDS[f & 3]` [observed: `t06_const_table.ts`].

Fact. $|KINDS| = 4$ at each entry of the function.

Where TF stops. The store location of the literal has a contract that admits every store of string cells. A transition that shortens it is admissible, so the fact is not $\Box_\top$. The fact holds because no code writes that store, which TF cannot state without ownership.

> **Definition 15** (Immutable store and immutable content)**.** A store location s has an *immutable store* when its contract admits one store σ. It has *immutable content* when, besides, each $l \in cells(\sigma)$ has a contract that admits one value. The cell of a module **`const`** binding has the contract that admits the value it is initialised with. An array literal held in a module **`const`** binding of a **`readonly`** array type is allocated with immutable content.

Example: in the worked program, cell 0 holds k for good, the store location 1 holds $\langle 2, 3, 4\rangle$ for good, and the locations 2, 3 and 4 hold their strings for good. So $F_2 = (|KINDS| = 3)$ and $\mathsf{every}(KINDS, 3, v \text{ is a string})$ are $\Box_\top$, which is what line 30 needs at every entry of `lastKind` [proved on paper: Section 11].

Sites. `prepareWithSegments` (142), `hasProperty` (234), `needsComplexTextPath` (89, 90).

Evidence. `kindName` is refused; `kindNameLocal`, the same literal declared in the function, is verified [observed: `t06_const_table.ts`].

7.2 P5: bounds held by the contents of generated tables (16 sites)

Program. A module cell **`let`** `rows: Uint8Array | null = null` is set by `rows ??= decode()`, where `decode` copies the bytes of a packed string into a new `Uint8Array`. The function reads `t[t[state]!]!`. With the table cached in the cell, the read is first blocked by P16; with the table decoded at each call, it is refused at `t[t[state]]` [observed: the case `bounds-pattern-generated-table-refused`].

Fact. $\mathsf{every}(t, \mathsf{view}(t), 0 \le v < \mathsf{view}(t))$ for the decoded table t.

Where TF stops. Two things are missing. First, no step establishes the fact: the decoder copies bytes without testing them. A check in the decoder establishes it, or an assumption that the generated data meets its generator's contract does (Section 12). Second, the element cells of the table are value locations, written by the decoder after allocation, and any later step may write them. Definition 15 fixes content at allocation, and a decoded table gets its content after allocation.

> **Definition 16** (Sealed content)**.** A *seal* of an array z at a step is the statement that, from that step on, the store location of z and every cell its store lists have the contract that admits the contents they hold at that step.

Example: `decode` seals `out` at its return. From then on $\mathsf{every}(out, \mathsf{view}(out), v < states)$, once checked, is $\Box_\top$. A seal changes a location's contract, which TF excludes (TF §10, *Fixed contracts*). Its soundness is [conj.] (Section 11).

Sites. In `line-breaks.ts` (`unpackTable`, `getClass`, `unpackClassRuns`, `unpackStateRows`, `markRuleBoundaries`), 12 in all; 2 in `gecko-line-breaks.ts`; 1 in `getWebKitGenericFamilies`; 1 in `getScriptFacts`. Each index is an entry or the length of a decoded table.

Evidence. Constant-stride rows verify [observed: `t09_arith.ts`, `goodStride`]. No program with a decoded table is run.

7.3 P16: a cache location initialised at its first use (4 sites)

Program. **`let`** `bmp: Uint16Array | null = null;` and **`const`** `table = (bmp ??= new Uint16Array(65536)); return table[cp]!;` under $0 \le cp < 65536$ [observed: `t07_lazy_cell.ts`].

Fact. $bmp \doteq$ **`null`** $\vee\ \mathsf{view}(bmp) = 65536$ at every entry (Definition 4).

Where TF stops. Within one invocation (INIT) keeps facts across `??=` (TF §8, *Write-once cache*). Across invocations the fact is a property of the cell's contents, and the contract `Uint16Array | null` does not fix it.

> **Definition 17** (Refined cell contract)**.** A module binding b may state a *refinement* $I_b(v)$, a formula of one value v that reads, besides v, only immutable locations. The contract of the cell l_b is the set of values of b's type that satisfy I_b. Each write to b proves I_b of the written value. Each read of b assumes it.

Example: $I_{bmp}(v) = (v \doteq$ **`null`** $\vee\ \mathsf{view}(v) = 65536)$. The initial value **`null`** satisfies it, and the write in `??=` writes a typed array of length 65536. The read `table[cp]` follows, since a typed array's length is immutable. The same

definition applies to a field written once by `??=`, such as `measurement.kanaKerning`.

Sites. `hasProperty` (232), `shouldBreakFast` (492), `nextBreakablePosition` (722), `getKanaKerning` (955). For the first three the table comes from `unpackTable`, so its length is also a P5 fact.

Evidence. `bitsAt` is refused; `bitsFresh`, the table allocated where it is read, is verified [observed: `t07_lazy_cell.ts`].

7.4 P20: a type invariant of a value read back from a Map (1 site)

Program. `type ClassTable = { readonly index: Uint16Array }` with the invariant `this.index.length === 4352`; a function returns `cache.get(name)` of a `Map<string, ClassTable>`, or builds the table and sets it [observed: `t18_map_cache.ts`].

Fact. view$(t.index) = 4352$ for the value t that `cache.get(name)` returns.

Where TF stops. TF has no invariant of a type. A value read from a native's result carries its type and no further fact.

> **Definition 18** (Type contract)**.** A type T may state an invariant $I_T(v)$, a formula of one value v that reads only immutable locations. The contract T is the set of values of type T that satisfy I_T. Each step that gives a value the type T proves I_T of it. Each read of a location of contract T assumes it.

Example: with the field `index` of `ClassTable` declared `@stable`, $I_{ClassTable}(v) = (\mathsf{view}(v.index) = 4352)$ reads only immutable locations. `build` gives its object literal the type `ClassTable` and proves the invariant there. `cache.get(name)` returns a value of type `ClassTable` or **`undefined`**, by the contract of `Map`, so it returns a value that meets the invariant or **`undefined`**.

Sites. `markRuleBoundaries` (361), reading `overrides.categories[i]` below `overrides.chars.length` of rules cached in a `Map`.

Evidence. `tableOf` is refused at its return; `classOfBuilt`, the record its builder returns, is verified [observed: `t18_map_cache.ts`].

8 Richer terms

TF's atoms are linear, quantify one array at a time, and name elements only through every. The five patterns here need five term forms. Each is class (a).

8.1 P9: a product of two variables in an index (7 sites)

Program. `rows[state * width]` under `rows.length === states * width` and $0 \le state < states$ [observed: `t09b_product.ts`, refused at the source gate].

> **Definition 19** (Row table)**.** A term may be a product $x \cdot y$ of two variables. The atom $\mathsf{table}(z, n, w)$ holds when $n, w \in \mathbb{N}$ and $|z| = n \cdot w$ (for a typed array, $\mathsf{view}(z) = n \cdot w$). Its rule (Row) is
>
> $$\mathsf{table}(z, n, w) \wedge 0 \le r < n \wedge 0 \le c < w \;\vdash\; 0 \le r \cdot w + c < |z|.$$

Example: with $\mathsf{table}(rows, states, width)$, `row = state * width` and $3 + category < width$, the read `rows[row + 3 + category]` is below the length. The support of $\mathsf{table}(z, n, w)$ is that of $|z|$.

Sites. `parseGraphemeRules` (46, 47, 48), `findGraphemeEnds` (85), `markRuleBoundaries` (369, 373, 382). Each also needs the state read from the table to be below `states`, a P5 fact.

8.2 P10: equal counts computed by two functions of one input (7 sites)

Program. `ends(text)` and `widthsOf(text)` each push one entry per non-space code unit of `text`; the caller reads `w[k]` for $k < |e|$ [observed: `t12_uf.ts`].

Fact. $|e| = |w|$.

> **Definition 20** (Named count)**.** A *count* g is declared by a prefix recurrence on strings: $g(x, 0) = 0$ and $g(x, i+1) = g(x, i) + \beta(x, i)$ for $i < \mathsf{strLength}(x)$, where $\beta(x, i) \in \{0, 1\}$ is a condition on the code unit of x at i. It names $g(x) = g(x, \mathsf{strLength}(x))$. Terms may contain $g(x)$ and $g(x, t)$ for a variable x and a linear term t.

Example: $\beta(x, i) = 1$ when the code unit at i is not 32. Both builders prove the loop invariant $|out| = g(text, i)$, and so the post $|r| = g(text)$; the caller derives $|e| = g(text) = |w|$. The support of $g(x)$ is $\emptyset$.

Sites. 6 in `measureAnalysis`, among them `fitAdvances[k]` under `k < extraSpacings.length`; 1 in `prepareRichInline` (523). Here g counts graphemes. Whether pretext's grapheme count has the form of Definition 20 is not checked [conj.].

8.3 P12: a pointwise relation of two arrays (5 sites)

Program. Under `forall k in fits: extras[k] === null || extras[k].length === fits[k].length`, the read `e[g]` with `e = extras[i]` and `g < fits[i].length` [observed: `t04_pointwise.ts`, refused at the source gate].

> **Definition 21** (Indexed every)**.** The atom $\mathsf{every}_k(t, \varphi)$ holds when, for each k with $0 \le k < t$, φ holds and every read in φ is defined. The formula φ may read, for several arrays x, the element $x[k]$, the length $|x[k]|$ of an element that is an array, a field $x[k].f$, and the guard $x[k] \doteq \texttt{null}$. Two regions name what it reads: $z[*].f$ denotes the locations $u(f)$ of the objects u held by the cells the store of z lists, and $z[*].\mathsf{store}$ denotes the store locations of the arrays held by those cells.

Example: $\mathsf{every}_k(|fits|,\ extras[k] \doteq \texttt{null} \vee |extras[k]| = |fits[k]|)$ bounds `e[g]` by `g < fits[i].length`. Its support is $\{fits[*] : \mathsf{keep},\ extras[*] : \mathsf{keep},\ fits[*].\mathsf{store} : \mathsf{keep},\ extras[*].\mathsf{store} : \mathsf{keep}\}$. TF's $\mathsf{every}(z, t, \varphi)$ is $\mathsf{every}_k(t, \varphi[z[k]/v])$ with one array. The 2 sites of P3 at line 1185 need $\mathsf{every}_k(t, fragments[k].itemIndex < n)$, of support $\{fragments[*] : \mathsf{keep},\ fragments[*].itemIndex : \mathsf{keep}\}$.

Sites. `countPreparedLines` (376), `walkPreparedComplexLines` (1027), `stepPreparedSimpleLineGeometry` (1134), `getPartWidth` (1001), `getWebKitParagraphBreaks` (704). The last relates $starts[k+1] - starts[k]$ to $\mathsf{strLength}(items[k].text)$.

8.4 P17: an index into the element another index selects (3 sites)

Program. `fits[c.segment]![c.grapheme]!` under `c.grapheme < fits[c.segment].length` [observed: `t11_cursor.ts`, refused at the source gate].

> **Definition 22** (Indexed path)**.** Paths are $\pi ::= x \mid \pi.f \mid \pi[t]$ with t a linear term. The read of $\pi[t]$ is the element at index t of the array that π reads; it fails at a hole or past the end. The region of the read of $\pi[t]$ is the store location of that array with the cell at index t.

Example: $c.grapheme < |fits[c.segment]|$ reads the fields of `c`, the store of `fits`, the cell at $c.segment$, and the store of the array that cell holds. Its support gives each of these keep, but the last, which tolerates grow.

Sites. `stepPreparedSimpleLineGeometry` (1134, 1134) and `getPartWidth` (998).

8.5 P19: a running sum of element lengths (2 sites)

Program. A loop `for (let i = 0, at = 0; i < segments.length; at += segments[i++]!.length)` reads `offsets[at]`, with $\mathsf{view}(offsets)$ the sum of the segments' lengths, the view being allocated at that sum. The read is refused [observed: the case `bounds-pattern-running-sum-refused`].

> **Definition 23** (Element sum)**.** For an array z, terms i, j and a term τ of one element, $\mathsf{sum}(z, i, j, \tau) = \sum_{i \le k < j} \tau(z[k])$. Linear atoms may contain it. Its rules are $\mathsf{sum}(z, j, j, \tau) = 0$ and $\mathsf{sum}(z, i, j, \tau) = \tau(z[i]) + \mathsf{sum}(z, i+1, j, \tau)$ for $i < j$.

Example: the loop invariant $at + \mathsf{sum}(segments, i, |segments|, \mathsf{strLength}) = \mathsf{view}(offsets)$ with each element nonempty gives $at < \mathsf{view}(offsets)$ while $i < |segments|$. Its support is $\{segments[*] : \mathsf{keep}\}$: a string element is a value.

Sites. `findSegmentSources` (1270, 1271).

9 Rules TF has and TScert lacks

Each pattern here is class (c): the fact is stated, and TF derives it, from rules TF states or from posts sound for the step.

9.1 P7: a fact over a local closure's captures (10 sites)

Program. `const glyphs = [...4 objects ...];` and a closure `group(from, to)` that reads `glyphs[i]!.advance` for $from \le i < to$, called as `group(0, 2) + group(2, 4)` [observed: `t19_closure.ts`].

In TF. `glyphs` is a `const` that no closure writes, so it is a variable (TF §2). The closure's precondition $|glyphs| = 4 \wedge 0 \le from \wedge to \le 4$ names it free, Call substitutes the parameters only, and Frame carries $|glyphs| = 4$ from the closure's creation to each call [paper: TF §2, Call, Frame].

Sites. `getHanKerningTrims` (212, 213), `getFontData` (140 four times, 142), `prepareRichInline` (577, 586, 620).

Evidence. `groups` is refused; `inline`, the same reads in the function's own body, is verified [observed: `t19_closure.ts`].

9.2 P11: a string's element read below its length (5 sites)

Program. `for (let i = 0; i < text.length; i++) out = text[i]!;` [observed: `t08_strings.ts`].

In TF. The fact $0 \le i < \mathsf{strLength}(text)$ is over variables and a string length, invariant by Definition 3, and the loop test establishes it. The step needs its post.

> **Proposition 24** (String read)**.** *[proved on paper]* $\{0 \le i < \mathsf{strLength}(s)\}\ y = s[i]\ \{y \text{ is a string} \wedge \mathsf{strLength}(y) = 1\}$.

Proof. For a string and an integer index below its length, JavaScript's indexed read yields the one-unit string of the code unit at that index. □

Sites. `segmentAtLineBreaks` (390, 396), `getWebKitLineRules` (787), `measureKanaKerning` (963, 963).

Evidence. `badElement` is refused; `goodCode`, a `charCodeAt(i)` in the same loop, is verified [observed: `t08_strings.ts`].

9.3 P13: a typed-array prefix filled with a postfix counter (5 sites)

Program. `for (let i = 1; i <= n; i++) ends[count++] = i;` for `ends: Int32Array` with $n \le \mathsf{view}(ends)$, returning `count`, with the prefix fact stated as the post. The loop's back edge is refused: the fact over the parameter `ends` is not extended across the write. The same loop over a view allocated in the function is verified [observed: the cases `bounds-pattern-postfix-prefix-refused` and `bounds-pattern-postfix-prefix`].

Fact. $\mathsf{every}(ends, count, 0 < v \le \mathsf{strLength}(text))$, established by the writes and returned in a post that names `count`.

> **Proposition 25** (Extending an every)**.** *[proved on paper] Let φ be a formula of one element whose other terms are invariant, and t a variable.*
>
> *(a) For a typed array z:* $\{\mathsf{every}(z, t, \varphi) \wedge 0 \le t < \mathsf{view}(z) \wedge \varphi(v)\}\ z[t] = v\ \{\mathsf{every}(z, t+1, \varphi)\}$.
>
> *(b) For an array z:* $\{\mathsf{every}(z, |z|, \varphi) \wedge \mathsf{dense}(z, |z|) \wedge \varphi(v)\}\ z.\mathtt{push}(v)\ \{\mathsf{every}(z, |z|, \varphi) \wedge \mathsf{dense}(z, |z|)\}$.

Proof. (a) The write changes the one element cell at index t. The cells at the indices below t are other cells of the same store, so they keep their contents, and the cell at t holds v. (b) The push changes the store of z by one new cell at index $\mathsf{old}(|z|)$, holding v, and leaves the other cells. In both cases φ reads nothing else that changes. □

With the renaming $count' = count + 1$ of the postfix increment, (a) and Loop keep the fact. In the worked program, (b) at line 10 keeps $\mathsf{every}(lists.kinds, |lists.kinds|, 0 \le v < 3)$, since the pushed value is 0, 1 or 2.

Sites. `setupClusterBoundaries` (353), `getUnspacedGraphemes` (263), `prepareRichInline` (698, 699, 700).

> *Remark* 26 (Establishing an every). The analysis attributes P13 to Loop. Loop keeps the fact; Proposition 25 establishes it at each write. TF states neither part of that proposition. The pattern is class (c).

9.4 P18: a guarded fact through two fields (2 sites)

Program. A type `Prepared` with `@stable` fields `widths` and `items: Items | null`, `Items` with a `@stable` field `insideExtras: number[] | null`, and the invariant $g \vee |p.items.insideExtras| = |p.widths|$ with $g = (p.items \doteq \texttt{null} \vee p.items.insideExtras \doteq \texttt{null})$ [observed: `t05_depth2_guard.ts`].

In TF. rigid($p.items$) holds, and under $\neg(p.items \doteq \texttt{null})$ so does rigid($p.items.insideExtras$). So g is invariant, and law (d) of TF's Proposition *Laws*, $g \vee \Box_\Phi F \vdash \Box_\Phi(g \vee F)$, moves the guard under the modality. In `badNoCall` the loop body only reads, its effect is $\emptyset$, and the fact is kept [paper: Proposition *Laws* (d), (Rigid), Loop].

Sites. `stepPreparedSimpleLineGeometry` (1138, 1138).

Evidence. `bad` and `badNoCall` are refused; `good`, a guard of one field, is verified [observed: `t05_depth2_guard.ts`].

9.5 P21: shift left and bitwise or (0 sites as first blocker)

Program. `values[(hi << 8) | (c & 0xff)]` under $0 \le hi < 4352$ and view($values$) = 1114112 [observed: `t09_arith.ts`].

> **Proposition 27** (Shift and or)**.** *[proved on paper] For integers* $0 \le a < 2^{23}$ *and* $0 \le d \le 255$, `(a << 8) | d` *equals* $256a + d$.

Proof. $256a < 2^{31}$, so the 32-bit shift is exact and its low 8 bits are zero. The or with $d < 256$ sets those bits to d. □

With $d = c \,\&\, 255 \in [0, 255]$, the index is $256\,hi + d < 4352 \cdot 256 = 1114112$. The post is linear.

Sites. None as first blocker; one as second blocker, in `getClass` (138), first blocked by P5.

Evidence. `badBits` is refused; `goodShift`, `index[c >> 8]`, is verified [observed: `t09_arith.ts`].

10 The classification

The table gives, for each pattern, its sites, the analysis's class, the class this note finds (Definition 2), and what it needs.

cap.	pattern	sites	analysis	found	needs
1	P1	50	(b)	(b)	47: Def. 5; 3: Def. 6
2	P2	31	(c)	(c)	Call with a two-state post
2	P8	10	(c)	(c)	Prop. 8
2	P14	4	(c)	(c)	native posts; Def. 3 for 1
3	P3	19	(c)	17 (c), 2 (a)	Tolerant frame; 2: Def. 21
3	P4	18	(b)	(c)	a regional callee effect, (Sep), (Rigid)
3	P6	12	(b)	(b)	Def. 13
4	P5	16	(d)	(d)	Def. 16, and a check or trust
4	P15	4	(d)	(d)	Def. 15
4	P16	4	(d)	(d)	Def. 17
4	P20	1	(d)	(d)	Def. 18
5	P9	7	(a)	(a)	Def. 19
5	P10	7	(a)	(a)	Def. 20
5	P12	5	(a)	(a)	Def. 21
5	P17	3	(a)	(a)	Def. 22
5	P19	2	(a)	(a)	Def. 23
6	P7	10	(c)	(c)	TF §2, Call, Frame
6	P11	5	(c)	(c)	Prop. 24, Def. 3
6	P13	5	(c)	(c)	Prop. 25, Loop
6	P18	2	(c)	(c)	Laws (d), (Rigid)
6	P21	0	(c)	(c)	Prop. 27

By class, the 215 blocked sites are: (a) 26, (b) 62, (c) 102, (d) 25. The analysis's counts are (a) 24, (b) 80, (c) 86, (d) 25. The difference is P4 (18 sites from (b) to (c)) and 2 sites of P3 (from (c) to (a)). The patterns P11 and P14 are class (c) only with Definition 3.

11 Soundness of the extensions

Each statement below says what makes one extension sound in TF's model: which properties of admissible transitions it uses, and what must be checked where. A statement marked [proved on paper] carries its argument.

String length (Def. 3) [proved on paper] An atom over variables and string lengths is invariant. Argument: strings are scalars, so strLength(x) depends on $\rho(x)$ alone; a step extends ρ and changes none of its bindings (TF §2). No (Tn) is used.

Complement and object regions (Def. 5) [proved on paper] For a variable x, the effect {all-values $\setminus$ $x.*$: any, all-stores : grow} is rigid. Argument: by (T3) $x.*$ denotes the same set in every heap. By admissibility a location keeps its contract, hence its class. A location of h changed by the first of two steps and again by the second lies, in both, in all-values $\setminus$ $x.*$ or in all-stores; transitivity follows as for TF's class regions, using (T1). For a path π that is not rigid, $\pi.*$ is read in each step's own heap and the effect is not rigid in general.

Bounded effect variable (Def. 6) [proved on paper] If a derivation of f's triple uses ε_p only as the effect of the calls of p and through $\varepsilon_p \sqsubseteq \Psi$, then replacing ε_p by a rigid $\Psi' \sqsubseteq \Psi$ whose regions rebase to themselves gives a derivation of the instance. Argument: by induction on the derivation. Frame at a call of p needs $\square_{\varepsilon_p} R$, which the derivation obtained from $\square_{\Psi} R$ by law (c); law (c) gives $\square_{\Psi'} R$ the same way. Loop and Seq-Eff need the effect rigid and its regions rebased to the entry; Ψ' is rigid, and its regions are class regions, kind regions, or regions at rigid paths, which rebase to themselves (TF §6, *Rebasing*). With the argument `undefined`, each call of p is skipped by its guard (`visit !== undefined` or `?.`) or throws before any change, so the instance $\emptyset$ bounds it. It uses (T1). What must be checked: at each call, $\Psi' \sqsubseteq \Psi$; for a visitor passed by code outside the analysed sources, this is assumed.

Kind regions (Def. 13) [proved on paper] all-fields, all-cells, all-elements and all-stores denote pairwise disjoint sets, and an effect over them is rigid. Argument: TF partitions dom h into $F \uplus C \uplus E \uplus S$, and a location's kind is fixed by what allocated it: an own key, a binding, an array element or an array. The rigidity argument of TF's class regions applies kind by kind. Respect by kind needs K = true. No (Tn) beyond (T1) is used.

Immutable store and content (Def. 15) Two statements. [proved on paper] A fact that reads only locations with single-value contracts is $\square_{\top}$: by (T4) each of those locations holds the same contents in every admissible successor. [conj.] Every step of the program respects the contracts given to a module literal: the literal is allocated with its contents, so the contracts hold at allocation; afterwards, the store or a cell is written only through a reference of a mutable array type, and a `readonly` array type is not assignable to a mutable one. This rests on the absence of casts and of `any` in the sources, and on natives that receive the array not writing it.

Sealed content (Def. 16) [conj.] A seal is sound when, at the sealing step, no reference to the array or its cells exists at a mutable type outside the sealing function's result. A seal narrows a location's contract after allocation. TF's worlds keep every contract (TF §3), so a seal needs worlds whose extension may narrow a contract, and a typing judgment that remains valid under narrowing. Neither is given here. After a seal, the statement of Def. 15 applies by (T4).

Refined cell contract (Def. 17) [proved on paper] If every write to b proves I_b and the initial value satisfies it, then $I_b(h(l_b))$ holds in every reachable heap. Argument: I_b reads besides v only immutable locations, so by (T4) its truth for a fixed v does not change. The contract of l_b is fixed at allocation, and every admissible successor meets it (TF §3). What must be checked: each write to b; all of them are in b's module, since an ES module's importers cannot assign its bindings.

Type contract (Def. 18) [proved on paper] If each step that gives a value the type T proves I_T, then every value of contract T in a reachable state meets I_T. Argument: as for the refined cell contract, with (T3) for the field locations I_T reads through and (T4) for their contents. What must be checked: each construction of a T, each assignment that types a value of another type at T, and each native whose result has type T; the last is an assumption about the native (here `Map.get`).

Row table (Def. 19) [proved on paper] Argument: $r \cdot w + c \le (n-1)w + (w-1) = nw - 1 < |z|$. The atom reads what $|z|$ reads, so it inherits that support.

Named count (Def. 20) [proved on paper] g is a function of a string value, defined by recurrence on $i \le$ strLength(x), so $g(x)$ reads no location and atoms over it are invariant. A post $|r| = g(x)$ is proved by the

builder like any linear post. What must be checked: that the declared recurrence is the one each builder's loop follows.

Indexed every and element regions (Def. 21) [proved on paper] The stated support is a support. Argument: the truth of $\mathsf{every}_k(t, \varphi)$ is determined by the stores of the arrays x, the contents of the cells they list, and the field locations and element stores those contents lead to; the support keeps each. The regions $z[*].f$ and $z[*].\mathsf{store}$ are read in the heap a change starts from, as TF's regions are. It uses (T3).

Indexed path (Def. 22) [proved on paper] A fact through $\pi[t]$ is supported by keep on each field, store and cell its read passes through, and by the change the fact tolerates on the last store it reads. Argument: those locations determine the read. A growth of a store may list a new cell at a present index: TF's $\preceq$ bounds the length, the dense prefixes and the newness of listed cells, and does not fix the cell at an index. So a store whose cell the read selects needs keep.

Element sum (Def. 23) [proved on paper] The two rules are the definition of a finite sum. The support $\{z[*] : \mathsf{keep}\}$ and the supports of τ determine every summand.

Posts (Props. 8, 24, 25, 27) [proved on paper] Each is a sound two-state post of its step, by the step's semantics, and enters a proof as Q_e of checked transfer (TF §7). Every other fact crosses the step by Frame.

12 What is not covered

The sites not placed. 3 sites of U:

`analyzeText` (522, 524) The reads `scanStarts[k]` below `paragraph.atomic.length` and `sourceBreaks[after]` with `after = scanStarts[k] + 1`. They need $|\mathit{scanStarts}| = |\mathit{paragraph.atomic}|$ after a callee pushes into `scanStarts`, and $\mathsf{every}(\mathit{scanStarts}, |\mathit{scanStarts}|, v < \mathsf{strLength}(r))$ for the callee's result string r. Both are TF assertions with Definition 3; the first is a P2 post when it needs old. Neither is tested.

`readScriptRuns` (193) The read `openBrackets[opened + 1]` with `opened = openBrackets.lastIndexOf(x)`. It is in bounds because the array holds pairs and `x` occurs at no odd index. The parity $\exists m.\ |\mathit{openBrackets}| = 2m$ is a TF assertion. The fact at odd indices needs Definition 21 with a guard on k, and the post of `lastIndexOf` [conj.]. Neither is tested.

Untested programs. Every pattern's program has an observed verdict, each read on the branch `single-pass` at `b7c8a559`. The worked program's verdicts are in Section 3. The module-cell program of P6 is this note's and is not run. The 165 locally supported sites without a verified shape are [conj.]. The assignment of each pretext site to a pattern is [conj.], except where the minimal program matches the site's shape directly: P1, P2, P3, P11, P15, P16 and P18.

Trust decisions. Three facts enter as assumptions, not as proofs:

The bound on a visitor passed by the application The walkers are exported. The application passes `onLine`, and its effect is not analysed. The bound Ψ_{onLine} of Section 4 is an assumption about it.

Native length contracts The posts of `replace`, `split`, `map`, `subarray` and `slice` in Section 5, the post of `lastIndexOf`, and the type of `Map.get`'s result in Section 7. Each is a contract of a native, audited and assumed.

Generated tables The entries of the tables that pretext's generator packs meet that generator's contract: a back reference, a run length, a class or a state below its bound. The alternative to the assumption is a check in each decoder.

Outside the account. The 34 rewrite markers whose operand is no index read (a nullable binding, a field, a call result), and the rewritten source's other changes, are outside this note.